

U1-AT2 OUTPUT

1.

```
At2.py - C:/Users/bmahe/AppData/Local/Programs/Python/Python312/At2.py (3.12.3)
File Edit Format Run Options Window Help

# Hospital Shift Scheduling using Backtracking
doctors = ["D1", "D2", "D3"]
shifts = ["Morning", "Afternoon", "Night"]
assignment = {}

# Constraint checking function
def is_valid(doctor, shift):
    # Only one doctor per shift
    if shift in assignment.values():
        print(" ", shift, "already assigned")
        return False

    # D1 cannot work Night
    if doctor == "D1" and shift == "Night":
        print(" Constraint Failed : D1 cannot work Night")
        return False

    # D3 cannot work Morning
    if doctor == "D3" and shift == "Morning":
        print(" Constraint Failed : D3 cannot work Morning")
        return False

    # Check D2 before D3
    temp = assignment.copy()
    temp[doctor] = shift

    if "D2" in temp and "D3" in temp:
        order = {"Morning":1, "Afternoon":2, "Night":3}

        if order[temp["D2"]] >= order[temp["D3"]]:
            print(" Constraint Failed : D2 must work before D3")
            return False

    return True

# Backtracking Function

IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help

Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
==== RESTART: C:/Users/bmahe/AppData/Local/Programs/Python/Python312/At2.py ====
Hospital Shift Scheduling

Trying D1 -> Morning
Assigned: {'D1': 'Morning'}

Trying D2 -> Morning
Morning already assigned

Trying D2 -> Afternoon
Assigned: {'D1': 'Morning', 'D2': 'Afternoon'}

Trying D3 -> Morning
Morning already assigned

Trying D3 -> Afternoon
Afternoon already assigned

Trying D3 -> Night
Assigned: {'D1': 'Morning', 'D2': 'Afternoon', 'D3': 'Night'}

Final Valid Schedule

D1 -> Morning
D2 -> Afternoon
D3 -> Night
>>>
```

2.

```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help

==== RESTART: C:/Users/bmahe/AppData/Local/Programs/Python/Python312/At2.py ====
Initial Queue : [(0, 0), (0, 0)]

Expanding Node : (0, 0)
Manhattan Distance = 7
Queue : [(1, 0), (0, 1)]

Expanding Node : (1, 0)
Manhattan Distance = 6
Queue : [(0, 1), (2, 0)]

Expanding Node : (0, 1)
Manhattan Distance = 6
Queue : [(2, 0), (0, 2)]

Expanding Node : (2, 0)
Manhattan Distance = 5
Queue : [(0, 2), (3, 0)]

Expanding Node : (0, 2)
Manhattan Distance = 5
Queue : [(3, 0), (1, 2)]

Expanding Node : (3, 0)
Manhattan Distance = 4
Queue : [(1, 2), (4, 0), (3, 1)]

Expanding Node : (1, 2)
Manhattan Distance = 4
Queue : [(4, 0), (3, 1), (2, 2)]

Expanding Node : (4, 0)
Manhattan Distance = 3
Queue : [(3, 1), (2, 2), (4, 1)]

Expanding Node : (3, 1)
Manhattan Distance = 3
Queue : [(2, 2), (4, 1)]

Expanding Node : (2, 2)
Manhattan Distance = 3
Queue : [(4, 1), (2, 3)]

Expanding Node : (4, 1)
Manhattan Distance = 2
Queue : [(2, 3), (4, 2)]

At2.py - C:/Users/bmahe/AppData/Local/Programs/Python/Python312/At2.py (3.12.3)
File Edit Format Run Options Window Help

from collections import deque

# Grid (S = Start, G = Goal, X = Obstacle)
grid = [
    ['S', '.', '.', 'X', '.', '.'],
    ['.', 'X', '.', 'X', '.', '.'],
    ['.', 'X', '.', '.', '.', '.'],
    ['.', '.', 'X', 'X', '.', '.'],
    ['.', '.', '.', 'G', '.', '.']
]

rows = cols = 5
start = (0,0)
goal = (4,3)

moves = [(-1,0), (1,0), (0,-1), (0,1)]

queue = deque()
queue.append((start, [start]))

visited = {start}

print("Initial Queue :", list(queue))

while queue:
    (x,y), path = queue.popleft()

    print("\nExpanding Node :", (x,y))

    h = abs(x-goal[0]) + abs(y-goal[1])
    print("Manhattan Distance =", h)

    if (x,y) == goal:
        print("Goal Reached!")
        print("Optimal Path :", path)
        print("Total Cost :", len(path)-1)
        break

    for dx,dy in moves:
        nx = x + dx
        ny = y + dy

        if 0<=nx<rows and 0<=ny<cols:
```

```

Expanding Node : (4, 2)
Manhattan Distance = 1
Queue : [(2, 4), (4, 3)]

Expanding Node : (2, 4)
Manhattan Distance = 3
Queue : [(4, 3), (1, 4), (3, 4)]

Expanding Node : (4, 3)
Manhattan Distance = 0

Goal Reached!
Optimal Path : [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3)]
Total Cost : 7

```

Ln: 91 Col: 23

3.

The screenshot shows a Python IDE with two windows. The left window displays the output of a program, and the right window displays the source code.

Output Window (Left):

```

Robot at : (2, 4)
Current Cost : 6
Adjacent Cells:
(1, 4) Discovered
(3, 4) Discovered
(2, 3) Discovered
Knowledge Updated

Robot at : (4, 2)
Current Cost : 6
Adjacent Cells:
(3, 2) Discovered
(4, 1) Discovered
(4, 3) Discovered
Knowledge Updated

Robot at : (1, 4)
Current Cost : 7
Adjacent Cells:
(0, 4) Discovered
(2, 4) Discovered
Knowledge Updated

Robot at : (3, 2)
Current Cost : 7
Adjacent Cells:
(2, 2) Discovered
(4, 2) Discovered
(3, 1) Discovered
Knowledge Updated

Robot at : (3, 4)
Current Cost : 7
Adjacent Cells:
(2, 4) Discovered
(4, 4) Discovered
Knowledge Updated

Robot at : (4, 3)
Current Cost : 7

Survivor Found!
Optimal Path : [(0, 0), (1, 0), (2, 0), (3, 0), (3, 1), (4, 1), (4, 2), (4, 3)]
Minimum Cost : 7
>>>

```

Source Code Window (Right):

```

# ATC.py - C:\Users\omane\AppData\Local\Programs\Python\Python12\ATC.py (3.12.3)
File Edit Format Run Options Window Help

import heapq

# Grid
grid = [
    ['S', '.', '.', '.', 'X', '.'],
    ['.', 'R', '.', '.', 'X', '.'],
    ['.', 'X', '.', '.', 'X', '.'],
    ['.', '.', 'R', 'X', '.'],
    ['.', '.', '.', 'G', '.']
]

rows = cols = 5

start = (0,0)
goal = (4,3)

moves = [(-1,0), (1,0), (0,-1), (0,1)]

pq = []

heapq.heappush(pq, (0, start, [start]))

visited = {}

print("Starting Rescue Robot...\n")

while pq:
    cost, (x,y), path = heapq.heappop(pq)

    if (x,y) in visited and visited[(x,y)] <= cost:
        continue

    visited[(x,y)] = cost

    print("Robot at :", (x,y))
    print("Current Cost :", cost)

    if (x,y) == goal:
        print("\nSurvivor Found!")
        print("Optimal Path :", path)
        print("Minimum Cost :", cost)
        break

    print("Adjacent Cells:")

```

Ln: 244 Col: 0 | Ln: 69 Col: 0